

En esta práctica se pide la implementación de un conjunto de funciones y un programa principal para el manejo de matrices de 8x8 enteros. En concreto, las funciones a implementar son:

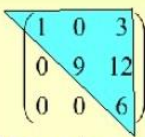
- rellenar una matriz pasada como parámetro con enteros en el rango [-20,20] usando la función aleatorio de la práctica anterior.
- imprimir una matriz por pantalla.
- multiplicar dos matrices que se pasan por parámetro dejando el resultado en la primera.
- convertir una matriz en triangular superior.
- rellenar una matriz como triangular inferior con valores en el rango [-20,20].
- rellenar una matriz identidad.

Ejemplos de Matriz Identidad / Unidad

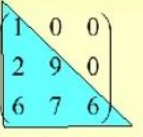
$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad B = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Matriz triangular

Matriz cuadrada en la que los elementos por debajo o por encima de la diagonal principal son ceros.



Triangular superior



Triangular inferior

El programa principal deberá definir una constante para el tamaño de la matriz y realizar las siguientes operaciones:

1. **Rellenar la matriz 1.**
2. Imprimir la matriz 1.
3. **Rellenar la matriz 2.**
4. Imprimir la matriz 2.
5. **Multiplicar la matriz 1 por la matriz 2.**
6. Imprimir la matriz 1.
7. **Convertir la matriz 1 en triangular superior.**
8. Imprimir la matriz 1.
9. **Convertir la matriz 2 en triangular superior.**
10. Imprimir la matriz 2.
11. **Multiplicar la matriz 2 por la matriz 1.**
12. Imprimir la matriz 2.-
13. **Rellenar como triangular inferior la matriz 1.-**
14. Imprimir la matriz 1.
15. **Multiplicar la matriz 2 por la matriz 1.-**
16. Imprimir la matriz 2.
17. **Rellenar como identidad la matriz 2.-**
18. Imprimir la matriz 2.
19. **Multiplicar la matriz 1 por la matriz 2.-**
20. Imprimir la matriz 1.

Una posible ejecución del programa sería:

```

-19  11  -4  -2  -7  -7  -16  -15
 6    6    9  17  -7  19    2    1
 4   -19   6  -3    7    1   17  -12
-15  -11  -20  -6  -16  -17  -6   17
18   -3   -7  -15  17   16   -9    0
13   18    1  -12  -14   6   11  -17
19   12   -9  -5  -14   -9   -7    0
 7   -1   -2   15  -5    9  -15  -14

14    7   -9  -15   16  -19   -9   -6
15   -5   12  -16  -11  -19  -11  -11
17   13  -11  20   -4  -17   -6  -19
-20   11  -15  16   19    1   18  -14
17  -19  -10   4   -8    4    5  -15
 8   18    1  -20   -6  -17    6   18
-10  -4  -12   15  -13  18   -9   11
-11  -16  -15   10    5   15   18   18

21    49   857  -281  -216  -204  -165  -370
-11   767  -286  -102  238  -664  211  -24
22   177  -378  459  -151  317  -245   -3
-1130 -626  273  261  240  1200  446  849
895  -114   58  -989  -51  -547  -95  202
596  484  537  -836  -374  -833  -976    6
153   33  362  -718  370  -470  -400  -34
 40  734  171  -454  527  -718  142  -383

21    49   857  -281  -216  -204  -165  -370
 0   767  -286  -102  238  -664  211  -24
 0    0  -378  459  -151  317  -245   -3
 0    0    0  261  240  1200  446  849
 0    0    0    0  -51  -547  -95  202
 0    0    0    0    0  -833  -976    6
 0    0    0    0    0    0  -400  -34
 0    0    0    0    0    0    0  -383

14    7   -9  -15   16  -19   -9   -6
 0   -5   12  -16  -11  -19  -11  -11
 0    0  -11  20   -4  -17   -6  -19
 0    0    0  16   19    1   18  -14
 0    0    0    0   -8    4    5  -15
 0    0    0    0    0  -17    6   18
 0    0    0    0    0    0   -9   11
 0    0    0    0    0    0    0   18

294  6055  13398 -12694 -4415 -21282  15306 -12334
 0  -3035  -3106  1042  -6281  9760  12858 -11249
 0    0  -4158  171  6665  36862  30987  23584
 0    0    0  4176  2871  7974  -2845  22178
 0    0    0    0  408  1044  -5144  3983
 0    0    0    0    0  14161  14192  -7200
 0    0    0    0    0    0  3600  -3907
 0    0    0    0    0    0    0  -6894

-20    0    0    0    0    0    0    0
-13   14    0    0    0    0    0    0
 7   -6    0    0    0    0    0    0
11  -12    9    7    0    0    0    0
12   -8   -9  -12  11    0    0    0
-15  -18  -12  19  -19   -4    0    0
 9   -8   20  15  -18   -8    0    0
15    6  -10  20  11  12   -9   17

80551 370654 610333-457326 -55389-105328 111006-209670
-226530-353092 325541 241748-609866-276924 101241-191233
190600-850228-1168901558080-925405-112336-212256 400928
267043 -60784-362623 547171 175243 257000-199602 377026
2685 42994-158910 17440 121057 84772 -35847 67711
-192687-411634 185908 337939-603715-256580 64800-122400
-26205 -52242 111070 -24140-107777 -75684 35163 -66419
-103410 -41364 68940-137880 -75834 -82728 62046-117198

 1    0    0    0    0    0    0    0
 0    1    0    0    0    0    0    0
 0    0    1    0    0    0    0    0
 0    0    0    1    0    0    0    0
 0    0    0    0    1    0    0    0
 0    0    0    0    0    1    0    0
 0    0    0    0    0    0    1    0
 0    0    0    0    0    0    0    1

-20    0    0    0    0    0    0    0
-13   14    0    0    0    0    0    0
 7   -6    0    0    0    0    0    0
11  -12    9    7    0    0    0    0
12   -8   -9  -12  11    0    0    0
-15  -18  -12  19  -19   -4    0    0
 9   -8   20  15  -18   -8    0    0
15    6  -10  20  11  12   -9   17

Process returned 10 (0xA)   execution time : 0.460 s
Press any key to continue.

```

SOLUCIÓN:

```
// includes y defines

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 8

// prototipos de las funciones

int aleatorio (int,int);
void rellenar(int matriz[N][N]);
void imprimir(int [N][N]);
void multiplicarmat1(int matriz1[N][N],int matriz2 [N][N]);
void triangular_superior (int matriz[N][N]);
void multiplicarmat2(int matriz2[N][N],int matriz1 [N][N]);
void triangular_inferior(int matriz[N][N]);
void identidad(int matriz[N][N]);
// main

int main(){
    int m1[N][N];
    int m2[N][N];
    rellenar(m1);
    printf("IMPRIMO MATRIZ 1: \n\n");
    imprimir(m1);
    printf("\n");
    rellenar(m2);
    printf("IMPRIMO MATRIZ 2: \n\n");
    imprimir(m2);
    printf("\n");
    multiplicarmat1(m1,m2);
    printf("IMPRIMO MATRIZ MULTIPLICAR DE MATRIZ 1 POR MATRIZ 2: \n\n");
    imprimir(m1);
    printf("IMPRIMO MATRIZ TRIANGULAR SUPERIOR DE LA MATRIZ 1: \n\n");
    triangular_superior(m1);
    imprimir(m1);
    printf("IMPRIMO MATRIZ TRIANGULAR SUPERIOR DE LA MATRIZ 2: \n\n");
    triangular_superior(m2);
    imprimir(m2);
    multiplicarmat2(m2,m1);
    printf("IMPRIMO MATRIZ MULTIPLICAR DE MATRIZ 2 POR MATRIZ 1: \n\n");
    imprimir(m2);
    printf("IMPRIMO MATRIZ TRIANGULAR INFERIOR DE LA MATRIZ 1: \n\n");
    triangular_inferior(m1);
    imprimir(m1);
    multiplicarmat2(m2,m1);
    printf("IMPRIMO MATRIZ MULTIPLICAR DE MATRIZ 2 POR MATRIZ 1: \n\n");
    imprimir(m2);
```

```
    identidad(m2);
    printf("IMPRIMO MATRIZ iDENTIDAD DE MATRIZ 2: \n\n");
    imprimir(m2);
    multiplicarmat1(m1,m2);
    printf("IMPRIMO MATRIZ MULTIPLICAR DE MATRIZ 1 POR MATRIZ 2: \n\n");
    imprimir(m1);
}

// Implemntación de las funciones

int aleatorio (int inf, int sup){
    int aux;
    if (inf > sup){
        aux = inf;
        inf = sup;
        sup = aux;
    }
    return(inf+rand()%(sup+1-inf));
}

void rellenar(int matriz[N][N]){
    int i,j;
    for(i=0;i<N;i++){
        for(j=0;j<N;j++){
            matriz[i][j] = aleatorio(-20,20);
        }
    }
}

void imprimir(int matriz [N][N]){
    int i,j;
    for(i=0;i<N;i++){
        for(j=0;j<N;j++){
            printf("%10d",matriz[i][j]);
        }
        printf("\n");
    }
}

void multiplicarmat1(int matriz1[N][N],int matriz2 [N][N]){
    int suma;
    int matrizAux[N][N];
    for (int a = 0; a < N; a++) { //es un valor fijo para las columnas de la segunda matriz, ya que si no lo
    utilizo al guardar el valor suma en la matriz aux ,se guardaría en otra posición que no es la primera.
        for (int i = 0; i < N; i++) {
            suma=0; //importante reiniciar la suma porque si no el valor de la suma se acumula.
            for (int j = 0; j < N; j++) {
                suma += matriz1[i][j] * matriz2[j][a]; //así es como se multiplican matrices
                //mirarlo con el dibujo de las matrices originales en papel
            }
        }
    }
```

```

        }
        matrizAux[i][a] = suma;
    }
}

for(int i=0;i<N;i++){
    for(int j=0;j<N;j++){
        matriz1[i][j] = matrizAux[i][j];
    }
}

void triangular_superior (int matriz[N][N]){
    for(int i=0;i<N;i++)
        for(int j=0;j<N;j++)
            if(i > j)
                matriz [i][j] = 0;
}

void multiplicarmat2(int matriz2[N][N],int matriz1 [N][N]){
    int suma;
    int matrizAux[N][N];
    for (int a = 0; a < N; a++) { //es un valor fijo para las columnas de la segunda matriz, ya que si no lo
        //utilizo al guardar el valor suma en la matriz aux ,se guardaría en otra posición que no es la primera.
        for (int i = 0; i < N; i++) {
            suma=0; //importante reiniciar la suma porque si no el valor de la suma se acumula.
            for (int j = 0; j < N; j++) {
                suma += matriz2[i][j] * matriz1[j][a]; //así es como se multiplican matrices
                //mirarlo con el dibujo de las matrices originales en papel
            }
            matrizAux[i][a] = suma;
        }
    }

    for(int i=0;i<N;i++){
        for(int j=0;j<N;j++){
            matriz2[i][j] = matrizAux[i][j];
        }
    }
}

void triangular_inferior(int matriz[N][N]){
    for(int i=0;i<N;i++)
        for(int j=0;j<N;j++)
            if(j > i)
                matriz [i][j] = 0;
}

void identidad(int matriz[N][N]){
    for(int i=0;i<N;i++){
        for(int j=0;j<N;j++){

```

```
        if( i!=j){
            matriz [i][j] = 0;
        }else{
            matriz[i][j]=1;
        }
    }
}
```